

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Навчально-науковий інститут комп'ютерних наук та інформаційних технологій
Кафедра комп'ютерної інженерії та програмування

ЗВІТ
з лабораторної роботи № 2
з дисципліни «Програмування»

Виконала: студентка групи КН-9266
Компанієць Світлана Костянтинівна
Перевірів: доцент кафедри КІП
Лисиця Дмитро Олександрович

Харків - 2026

Тема: Вступ до програмування. Основи debug процесу.

Мета роботи: ознайомитися з Git, створити приватний репозиторій для курсу та налаштувати базову структуру проєкту.

Хід роботи

1. Створено публічний репозиторій у системі Github за наступною назвою: **programing-kompaniiets** (рис.1):

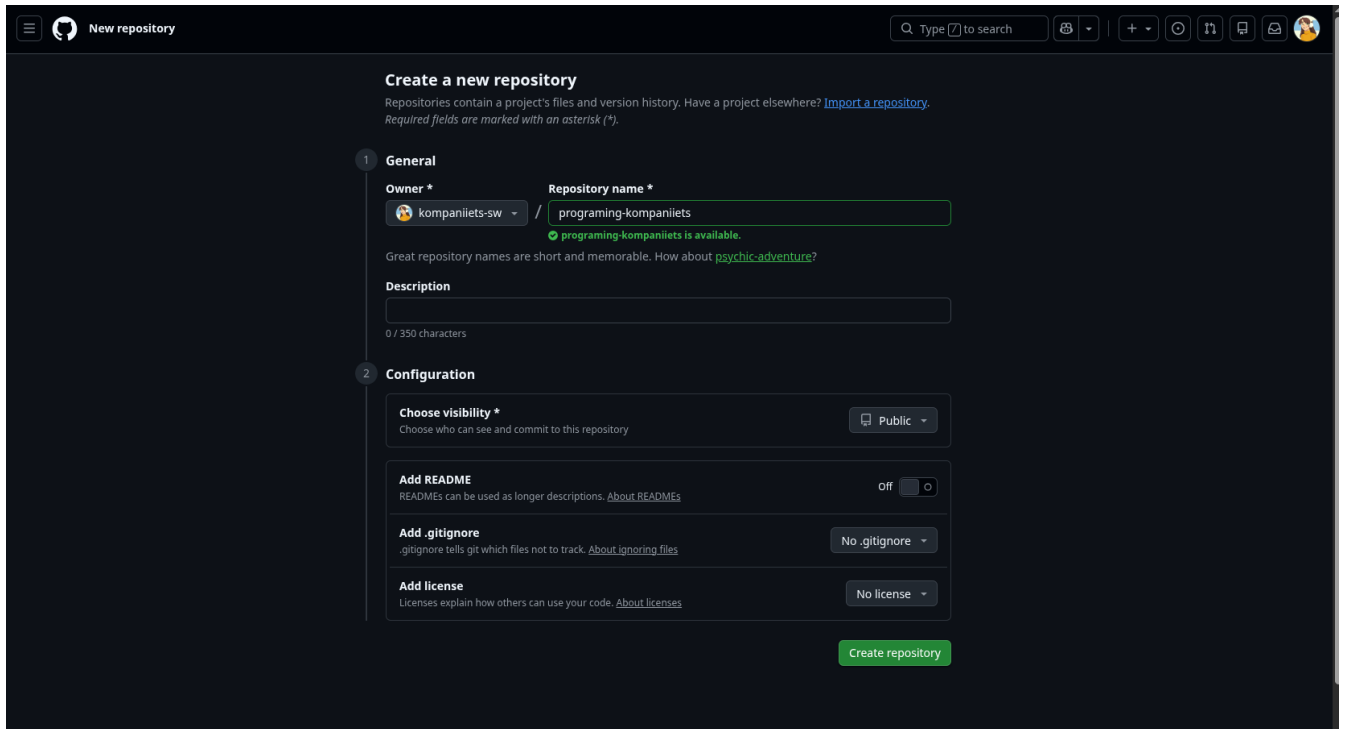


Рисунок 1 – Скриншот веб-сторінки github

2. Далі створений репозиторій було скопійовано на комп'ютер (рис.2):

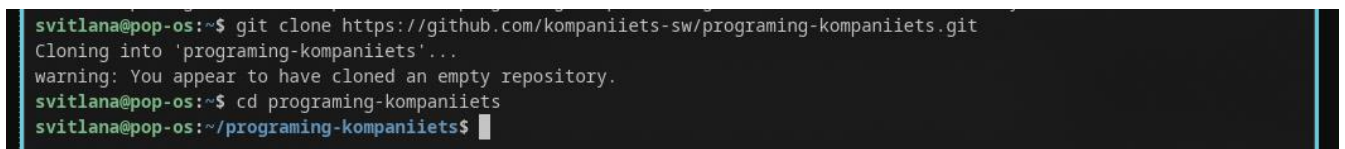
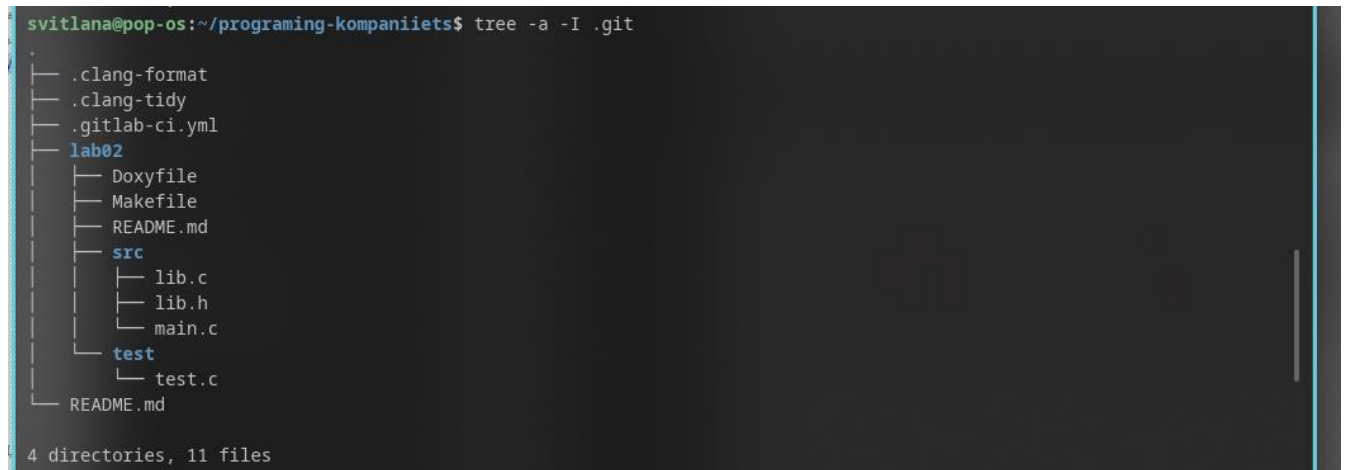


Рисунок 2

3. У клонований репозиторій було скопійовано файли з минулої лабораторної роботи за допомогою наступних команд (рис.3):

```
svitlana@pop-os:~/programing-kompaniiets$ mkdir -p lab02
svitlana@pop-os:~/programing-kompaniiets$ cp ~/sample_project/.clang-format .
svitlana@pop-os:~/programing-kompaniiets$ cp ~/sample_project/.clang-tidy .
svitlana@pop-os:~/programing-kompaniiets$ cp ~/sample_project/.gitlab-ci.yml .
svitlana@pop-os:~/programing-kompaniiets$ cp ~/sample_project/README.md .
svitlana@pop-os:~/programing-kompaniiets$ cp ~/sample_project/lab00/Doxyfile ./lab02/
```

```
svitlana@pop-os:~/programing-kompaniiets$ cp ~/sample_project/lab00/Makefile ./lab02/
svitlana@pop-os:~/programing-kompaniiets$ cp ~/sample_project/lab00/README.md ./lab02/
svitlana@pop-os:~/programing-kompaniiets$ cp -r ~/sample_project/lab00/src ./lab02/
```

A terminal window with a dark background and light blue text. The prompt is 'svitlana@pop-os:~/programing-kompaniiets\$'. The command 'tree -a -I .git' has been executed. The output shows a directory tree starting with a root directory containing files like '.clang-format', '.clang-tidy', and '.gitlab-ci.yml'. A subdirectory 'lab02' is highlighted in blue and contains 'Doxyfile', 'Makefile', 'README.md', and a subdirectory 'src'. The 'src' directory contains 'lib.c', 'lib.h', and 'main.c'. Another subdirectory 'test' is highlighted in blue and contains 'test.c'. A final 'README.md' file is at the root level. At the bottom, it says '4 directories, 11 files'.

```
svitlana@pop-os:~/programing-kompaniiets$ tree -a -I .git
.
├── .clang-format
├── .clang-tidy
├── .gitlab-ci.yml
├── lab02
│   ├── Doxyfile
│   ├── Makefile
│   ├── README.md
│   └── src
│       ├── lib.c
│       ├── lib.h
│       └── main.c
├── test
│   └── test.c
└── README.md

4 directories, 11 files
```

Рисунок 3 - Створена архітектура клонованої папки

4. Зафіксували зміни у локальному репозиторії за допомогою комміту Git. Для цього спочатку додали всі нові файли до індексу за допомогою команди `git add .`. Під час першої спроби виконати `git commit` виникла помилка ідентифікації користувача ("Author identity unknown"). Для її усунення було налаштували глобальні параметри імені та електронної пошти за допомогою команд `git config --global user.email "*пошта*"` та `git config --global user.name "*логін*"`. Після цього повторно зафіксували зміни командою `git commit -m "Initial copy of sample_project"` та перевірили стан репозиторію за допомогою `git status` (рис.4):

```

svitlana@pop-os:~$ cd ~/programing-kompaniiets
svitlana@pop-os:~/programing-kompaniiets$ git add .
svitlana@pop-os:~/programing-kompaniiets$ git commit -m "Initial copy of sample_project"
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'svitlana@pop-os.(none)')
svitlana@pop-os:~/programing-kompaniiets$ git config --global user.email "kompaniiets.switlana@gmail.com"
svitlana@pop-os:~/programing-kompaniiets$ git config --global user.name "kompaniiets-sw"
svitlana@pop-os:~/programing-kompaniiets$ git commit -m "Initial copy of sample_project"
[main (root-commit) 0bda2e8] Initial copy of sample_project
11 files changed, 390 insertions(+)
create mode 100644 .clang-format
create mode 100644 .clang-tidy
create mode 100644 .gitlab-ci.yml
create mode 100644 README.md
create mode 100644 lab02/Doxyfile
create mode 100644 lab02/Makefile
create mode 100644 lab02/README.md
create mode 100644 lab02/src/lib.c
create mode 100644 lab02/src/lib.h
create mode 100644 lab02/src/main.c
create mode 100644 lab02/test/test.c
svitlana@pop-os:~/programing-kompaniiets$ git status
On branch main
Your branch is based on 'origin/main', but the upstream is gone.
(use "git branch --unset-upstream" to fixup)

nothing to commit, working tree clean
svitlana@pop-os:~/programing-kompaniiets$

```

5. Для додавання нового типу тварини «Людина» у файлі `src/lib.h` оновили перелік `enum animal_type`:

```

/**
 * Тип тварини
 */

enum animal_type {
    PIG, /**< Свиня */
    COW, /**< Корова */
    DOG, /**< Собака */
    CAT, /**< Кіт */
    HUMAN, /**< Людина */
    ANIMAL_TYPE_COUNT /**< Кількість тварин */
};

```

Також у файлі `src/lib.c` додали обробку нового типу `HUMAN` у функції `get_animal_type_name`:

```
const char *get_animal_type_name(enum animal_type type)
{
    const char *result;

    switch (type) {
        case CAT:
            result = "Фікус";
            break;

        case DOG:
            result = "Монстера";
            break;

        case COW:
            result = "Заміокулькас";
            break;

        case PIG:
            result = "Кактус";
            break;

        case HUMAN:
            result = "Людина";
            break;

        default:
            result = "N/A";
    }

    return result;
}
```

Після відформатування коду за допомогою утиліти `clang-format -i src/*` проєкт було успішно зібрано командою `make` та виконано запуск скомпільованого файлу `./dist/main.bin` (рис. 5):

```
svitlana@pop-os:~/programing-kompaniiets/lab02$ ./dist/main.bin
Інформація про рослину №01: Фікус: висота = 120 см, кількість листків = 121 шт.
Інформація про рослину №02: Заміокулькас: висота = 65 см, кількість листків = 31 шт.
Інформація про рослину №03: Кактус: висота = 31 см, кількість листків = 57 шт.
Інформація про рослину №04: Монстера: висота = 122 см, кількість листків = 119 шт.
Інформація про рослину №05: Заміокулькас: висота = 14 см, кількість листків = 74 шт.
Інформація про рослину №06: Монстера: висота = 116 см, кількість листків = 79 шт.
Інформація про рослину №07: Монстера: висота = 9 см, кількість листків = 1 шт.
Інформація про рослину №08: Монстера: висота = 12 см, кількість листків = 83 шт.
Інформація про рослину №09: Кактус: висота = 41 см, кількість листків = 71 шт.
Інформація про рослину №10: Людина: висота = 54 см, кількість листків = 24 шт.
svitlana@pop-os:~/programing-kompaniiets/lab02$
```

Рисунок 5 - Внесені зміни у проєкт

6. Запустили виконуваний файл у відладнику lldb, встановили точку зупинки на функцію `get_animal_type_name`, далі перевірили значення змінних оточення та випробували команди навігації `s` (step) (захід усередину), `n` (next) (покрокове виконання) та `frame variable/v` (перегляд змінних) (рис.6):

```
svitlana@pop-os:~/programing-kompaniiets/lab02$ lldb ./dist/main.bin
(lldb) target create './dist/main.bin'
Current executable set to './dist/main.bin' (x86_64).
(lldb) b get_animal_type_name
Breakpoint 1: where = main.bin`get_animal_type_name + 7 at lib.c:17:10, address = 0x0000000000001177
(lldb) run
Process 11177 launched: './dist/main.bin' (x86_64)
Process 11177 stopped
* thread #1, name = 'main.bin', stop reason = breakpoint 1.1
  frame #0: 0x0000555555551177 main.bin`get_animal_type_name(type=COW) at lib.c:17:10
    14  const char *get_animal_type_name(enum animal_type type)
    15  {
    16      const char *result;
->  17      switch (type) {
    18          case CAT:
    19              result = "Фікус";
    20              break;
(lldb) frame variable
(animal_type) type = COW
(const char *) result = 0x00007ffffffded8 "@\xe2\xff\xff\xff\U0000007f"
(lldb) step
Process 11177 stopped
* thread #1, name = 'main.bin', stop reason = step in
  frame #0: 0x00005555555511bc main.bin`get_animal_type_name(type=COW) at lib.c:25:10
    22      result = "Монстера";
    23      break;
    24      case COW:
->  25      result = "Заміокулькас";
    26      break;
    27      case PIG:
    28      result = "Кактус";
(lldb) v
(animal_type) type = COW
(const char *) result = 0x00007ffffffded8 "@\xe2\xff\xff\xff\U0000007f"
(lldb) n
Process 11177 stopped
* thread #1, name = 'main.bin', stop reason = step over
  frame #0: 0x00005555555511c7 main.bin`get_animal_type_name(type=COW) at lib.c:26:3
    23      break;
    24      case COW:
    25      result = "Заміокулькас";
->  26      break;
    27      case PIG:
    28      result = "Кактус";
    29      break;
(lldb) v
(animal_type) type = COW
(const char *) result = 0x0000555555556034 "Заміокулькас"
(lldb)
```

Рисунок 6 - Процес відлагодження програми та аналіз змінних у дебагеру lldb

7. Зафіксували всі внесені зміни у сирцевому коді програми за допомогою комміту в локальний репозиторій Git, використавши команду `git add` . для додавання змінених файлів до індексу та `git commit -m` із відповідним повідомленням про додавання нового типу тварини (рис.7):

```
svitlana@pop-os:~/programing-kompaniets/lab02$ cd ~/programing-kompaniets
svitlana@pop-os:~/programing-kompaniets$ git add .
svitlana@pop-os:~/programing-kompaniets$ git commit -m " Add HUMAN animal type"
[main 2eeafee] Add HUMAN animal type
4 files changed, 11 insertions(+), 2 deletions(-)
create mode 100755 lab02/dist/main.bin
svitlana@pop-os:~/programing-kompaniets$ git status
On branch main
Your branch is based on 'origin/main', but the upstream is gone.
(use "git branch --unset-upstream" to fixup)

nothing to commit, working tree clean
```

```
svitlana@pop-os:~/programing-kompaniets$ git push
Username for 'https://github.com': kompaniets-sw
Password for 'https://kompaniets-sw@github.com':
Enumerating objects: 25, done.
Counting objects: 100% (25/25), done.
Delta compression using up to 4 threads
Compressing objects: 100% (22/22), done.
Writing objects: 100% (25/25), 10.84 KiB | 2.71 MiB/s, done.
Total 25 (delta 5), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (5/5), done.
To https://github.com/kompaniets-sw/programing-kompaniets.git
 * [new branch]      main -> main
svitlana@pop-os:~/programing-kompaniets$
```

Рисунок 7 - Команда `git push` виконана успішно

8. Дослідили режими компіляції програми з використанням відлагоджувальної інформації та без неї.

Головна відмінність між цими режимами полягає у наявності відлагоджувальних символів у кінцевому виконуваному файлі. При компіляції з прапорцем `-g` (Debug) компілятор додає до бінарного файлу спеціальні метки, які пов'язують машинні інструкції з конкретними рядками сирцевого коду та назвами змінних. Це дозволяє

відлагоджнику lldb зупинятися на потрібних рядках і показувати зрозумілі назви змінних під час розробки, проте сам файл стає більшим за розміром.

У режимі без прапорця -g (Release) ця додаткова інформація не додається, а код може додатково оптимізуватися компілятором. У результаті файл займає менше місця на диску та працює швидше, але повноцінне відлагодження стає неможливим, оскільки lldb замість рядків коду та імен змінних бачитиме лише машинні адреси пам'яті.

Контрольні запитання

1. Що таке процес відлагодження (debug)?

Це пошук, аналіз та усунення помилок у коді за допомогою покрокового виконання програми та перевірки стану її змінних.

2. Чим відрізняється бінарний файл з та без інформацією про відлагодження?

Відлагоджувальний файл містить символічні дані (назви змінних, функцій, номери рядків коду) і має більший розмір, тоді як файл без відлагоджувальної інформації оптимізований за розміром і швидкістю та не прив'язаний до сирцевого тексту.

3. Які базові утиліти необхідні для можливості відлагодження додатків?

Компілятор із підтримкою відлагоджувальних прапорів (gcc чи clang), сам відлагоджник (lldb або gdb) та аналізатори пам'яті (valgrind чи sanitizers).

4. Для чого необхідна утиліта lldb?

Це консольний відлагоджник середовища LLVM для покрокового виконання програм, аналізу пам'яті, контролю змінних і дослідження причин аварійного завершення коду.

5. Як дізнатися поточне значення змінної у відлагоднику?

У відлагоднику значення змінної дізнаються за допомогою команди `p назва_змінної` або `frame variable`.

6. Що таке точка зупинки (breakpoint)? Як її поставити?

Це мітка на певному рядку або функції для призупинення виконання програми. У lldb вона ставиться командою `b main.c:48` (на рядок) або `b main` (на функцію).

7. Що таке стек виклику? Як його подивитися?

Це структура даних із послідовністю ще не завершених функцій у порядку їх виклику. Переглянути його можна командою `bt`.

8. Що роблять наступні команди: Step In, Step Out, Step Over?

Step In (s) заходить у середину викликаної функції, Step Over (n) виконує поточний рядок без заходу у функції, а Step Out (finish) допрацьовує поточну функцію до кінця і повертає контроль у викликаючу.

Висновок. У ході лабораторної роботи було створено та налаштовано репозиторій Git, виконано роботу з основними командами `git add`, `git commit` та `git push`. Також було виконано компіляцію проєкту та внесено необхідні зміни. Отримано практичні навички роботи з відлагоджувачем LLDB, зокрема перегляду значень змінних і покрокового виконання програми. Додатково було досліджено різницю між компіляцією з інформацією для відлагодження та без неї. У результаті роботи закріплено базові навички роботи з Git, компіляції та налагодження програм.